```
log_b(N) = log_2(N) / log_2(b)
```

If `b > 2`, it's apparent that `logb(N) < log2(N)`, and therefore if we have a logarithmic factor in our algorithm's running time, and we manage to provide an implementation that instead of `log2(N)` steps will require `logb(N)`, while all other factors stay unchanged, then we will have provided a (constant-time) speed-up.

In section 2.9 we will further investigate how this applies to d-ary heaps.

2.9.2 Running time

So, the answer is yes, there is an advantage in tuning the heap's branching factor, but compromise is the key.

The insertion will always be quicker with larger branching factors, as we at most need to bubble up the new element to the root, with `O(logD(n))` comparisons and swaps at most.

The branching factor will, instead, affect deletion and priority update. If you recall the binary heap algorithms for popping elements, for each node we need to first find the highest-priority among all its children, then compare it to the element we are pushing down.

The larger the branch factor, the smallest the height of the tree (it shrinks logarithmically with the branching factor). But, on the other hand, the number of children to compare at each level also grows linearly with the branch factor. So, as you can imagine, a branching factor of 1000 wouldn't work